UNIVERSIDAD DE LA SIERRA JUÁREZ

CARRERA:
LICENCIATURA EN INFORMÁTICA

ASIGNATURA:
Calidad de Software

NOMBRE DEL TEMA:
Cuadro Sinóptico de Herramientas
de Calidad de Software

Profesor: M.I.T.I. Laura López Núñez

---

Alumno:
Yenisleydi Benitez Martinez
Semestre: Octavo

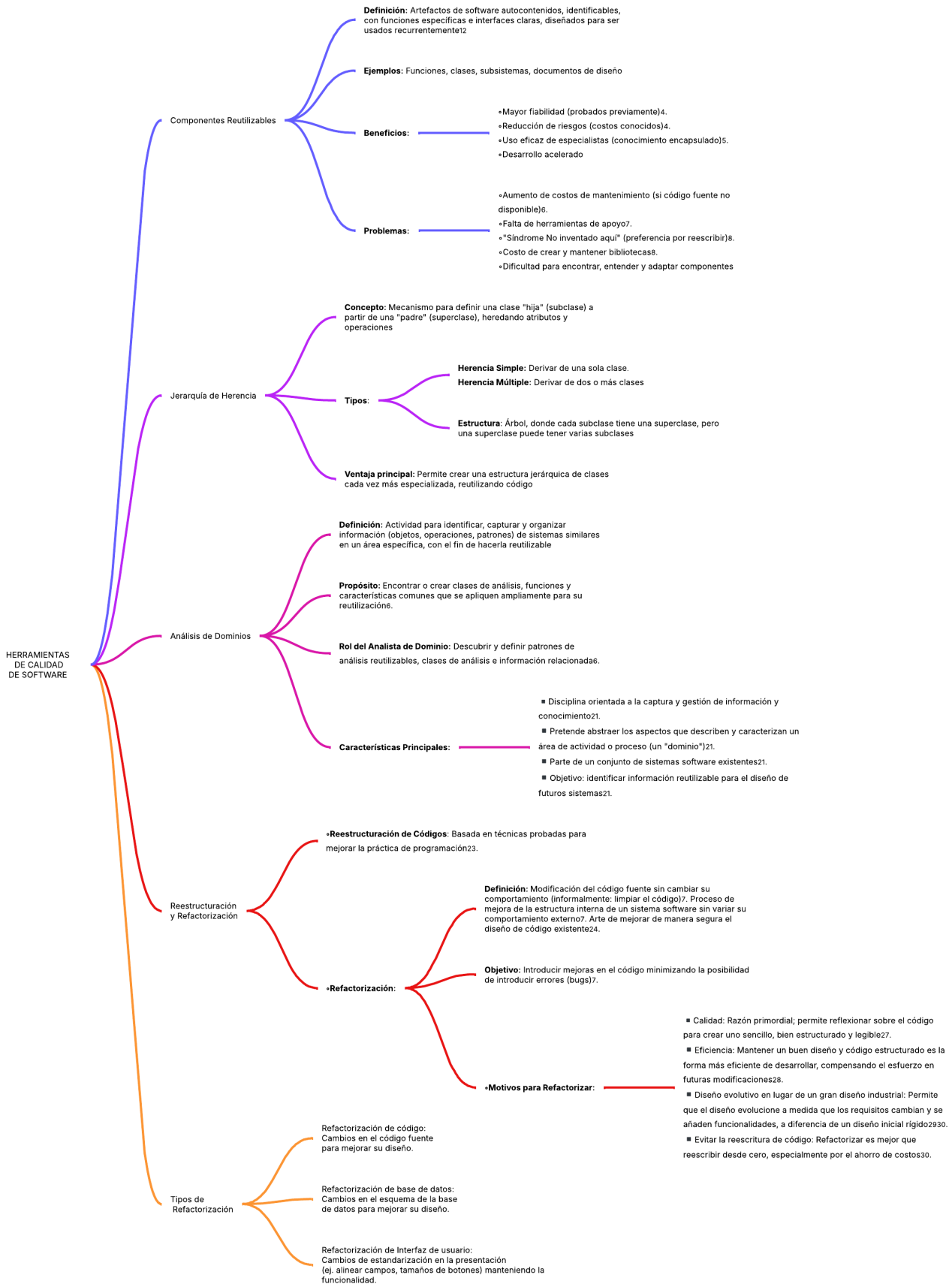
# Introducción

El presente documento tiene como objetivo principal presentar un cuadro sinóptico detallado sobre la Unidad VI: Herramientas de Calidad de Software, basándose en los materiales de estudio proporcionados. En el ámbito de la ingeniería de software, la calidad es un pilar fundamental para el éxito y la sostenibilidad de cualquier proyecto. Para alcanzar y mantener altos estándares de calidad, se recurre a diversas herramientas y metodologías que optimizan el proceso de desarrollo.

En este trabajo, se abordarán conceptos esenciales y estratégicos como los Componentes Reutilizables, que buscan maximizar el uso recurrente de software existente; la Jerarquía de Herencia, un mecanismo que permite la reutilización del código en la Programación Orientada a Objetos (OOP); el Análisis de Dominios, fundamental para identificar información reutilizable en áreas específicas; y la Reestructura y Refactorización, prácticas clave para mejorar la estructura interna del código sin alterar su comportamiento externo, contribuyendo a su claridad y mantenibilidad. Se explorarán también los Tipos de Refactorización.

La finalidad de este documento es ofrecer una visión organizada y comprensible de estas herramientas, destacando su relevancia y los beneficios que aportan al proceso de desarrollo de software, así como los desafíos asociados a su implementación.

# 1. Cuadro Sinóptico



# Conclusiones

Las herramientas de calidad de software analizadas en este documento son esenciales para la ingeniería de software contemporánea. La reutilización de componentes se erige como una estrategia clave para el desarrollo de sistemas, al permitir la construcción de software a partir de piezas ya existentes. Esta práctica conlleva múltiples beneficios, incluyendo una mayor fiabilidad del software al utilizar componentes previamente probados, una reducción significativa de riesgos en la estimación de costos, un uso más eficiente de los especialistas al encapsular su conocimiento, el cumplimiento de estándares mediante componentes predefinidos, y una notable aceleración en el desarrollo de sistemas. Sin embargo, la reutilización también presenta desafíos importantes, como el potencial aumento de los costos de mantenimiento si no se dispone del código fuente, la falta de herramientas de apoyo adecuadas, la resistencia de algunos ingenieros a reutilizar código ajeno (”síndrome de no inventado”), y las complejidades inherentes a la gestión y localización efectiva de componentes en bibliotecas. La jerarquía de herencia es un mecanismo fundamental en la Programación Orientada a Objetos que potencia intrínsecamente la reutilización del código. Permite que las clases especializadas hereden atributos y operaciones de clases más generales, generando estructuras de código más modulares, eficientes y fáciles de mantener.
Por otro lado, el análisis de dominios desempeña un rol crucial al sistematizar la identificación, captura y organización de información relevante para el desarrollo de software, con el objetivo explícito de hacerla reutilizable en la creación de nuevos sistemas. Es una disciplina que abstrae las características de un área específica para generar patrones aplicables en múltiples contextos.
Finalmente, la refactorización es una práctica indispensable en el ciclo de vida del software. Consiste en mejorar la estructura interna del código sin modificar su comportamiento externo. Los motivos para refactorizar son fundamentalmente la búsqueda de calidad, la eficiencia a largo plazo en el desarrollo, la facilitación de un diseño evolutivo que se adapta a requisitos cambiantes, y la evitación de la reescritura completa de sistemas. Es, en esencia, el arte de mejorar de manera segura el diseño de código existente, lo que resulta en un software más claro, menos redundante, más seguro, más legible y más fácil de mantener. Aunque presenta desafíos humanos y técnicos como la dependencia de la estructura de bases de datos o los cambios de interfaz, sus beneficios para la calidad y mantenibilidad del software la convierten en una práctica irremplazable.